

WEEK 2 DAY 1

1. Task 1:

In this task, I learned the difference between an agent, a chatbot, and a workflow. An agent can decide what to do next, use tools, look at the results, and continue through several steps. A chatbot mainly responds to the user, while a workflow follows steps that have already been defined by the developer. I also looked at what makes a system agentic: autonomy, tool use, multi-step planning, and the ability to react to new information or errors.

For the ReAct pattern, the process can be described simply as: User request -> LLM decides what to do -> tool call -> tool executes -> result is returned -> LLM looks at the result -> repeat if another step is needed -> final answer. I also noted that an agent is not always necessary. If a task can be solved with one prompt, a normal Python function, or a fixed sequence of API calls, using an agent can add unnecessary complexity, latency, and failure points.

2. Task 2:

For this task, I created two tools in Python: a calculator and a simulated weather lookup. The calculator accepts an arithmetic expression, while the weather tool accepts a city name and returns the weather data stored in the notebook. Each tool was described with a name, description, and input schema so the model knows what the tool is for and what arguments it should provide.

I then tested the tool-calling process with a single request. The model selected the appropriate tool, Python executed that function, and the result was prepared as a tool result and sent back to the model. Clear descriptions are important because the model uses them to decide which tool fits the request and how to construct its arguments.

3. Task 3:

Here I built the main agent loop from scratch in Python rather than using an agent framework. The program starts with the user's message, sends it to Gemini, checks whether Gemini requested a tool, runs the requested Python function, adds the tool result to the conversation, and sends the updated conversation back to Gemini. This continues until Gemini produces a normal final response.

I also added a `max_iterations` limit so the program cannot keep running forever if something goes wrong. For the multi-step test, I asked the agent to look up the weather in Lahore and Karachi and compare their temperatures. This demonstrated that the agent can make more than one tool call before producing its final answer.

4. Task 4:

In this task, I separated conversation memory from working memory. Conversation memory is the message history that is repeatedly sent to the model, including the original request, tool calls, and tool results. Working memory is the extra state maintained by the Python program while the

task is running, such as the current iteration number and which tools have been used.

I also added logging to the agent. While it runs, the notebook prints the current iteration, the tool being called, its arguments, and the observation returned by the tool. This makes it easier to see what the agent is actually doing and helps with debugging.

5. Task 5:

For the failure test, I deliberately asked the agent for weather information for a city that was not included in the simulated weather data. Instead of inventing weather information in the Python tool, the tool returned an error message saying that no stub data was available. The agent then received that result as an observation and had to respond based on what the tool actually returned.

This task also showed several common problems that can occur in an agent: an infinite loop, choosing the wrong tool, providing incorrect arguments, a tool or API error, asking for a capability that has not been implemented, and producing a result without actually using the required tool. The safeguards used in the notebook include a maximum iteration count, clear tool descriptions, input validation, exception handling, structured error results, and checking that tool-dependent information comes from an actual tool result.